

COMPSCI 589 – Final Project

Benchmarking Classical ML Algorithms on Four Datasets

Vishal Hanuman

Akshit [Last Name]

May 8, 2026

Note on Implementations

All machine learning algorithms were implemented from scratch. No machine learning libraries (e.g., scikit-learn, PyTorch) were used for training or prediction. `numpy` and `matplotlib` were used for numerical computation and plotting. `scikit-learn` was used *only* for dataset loading, not for training or prediction.

- **Datasets 1–2** (Digits, Parkinson’s): k-NN, Gaussian Naive Bayes, Random Forest, and Neural Network implementations by **Vishal Hanuman** (HW1–HW4).
- **Datasets 3–4** (Rice, Credit Approval): Implementations by **Akshit [Last Name]** (HWX).

1 Introduction

The goal of this project is to compare the performance of classical machine learning algorithms on four benchmark datasets by evaluating accuracy and F1-score under stratified 10-fold cross-validation, with systematic hyperparameter tuning.

2 Datasets

1. **Digits** – 1,797 instances, 64 numerical features, 10 classes (0–9). Loaded using the Digits loader from `scikit-learn`.
2. **Parkinson’s Disease** – 195 instances, 22 numerical features, binary (0 = healthy, 1 = Parkinson’s).
3. **Rice Grains** – 3,810 instances, 7 numerical features, binary (Cammeo vs. Osmancik).
4. **Credit Approval** – 653 instances, 6 numerical + 9 categorical features, binary (approved vs. not).

3 Experiments and Analyses

3.1 Dataset 1 – Hand-Written Digits

For the Digits dataset, I evaluated k-NN, Random Forest, and a Neural Network. All features are numerical pixel intensities, so no feature encoding was required. k-NN is a natural baseline because

digits with similar pixel layouts should be close in the normalized feature space. Random Forests were tested because they can learn nonlinear decision rules over pixel thresholds, while the Neural Network was included because it can combine many pixel-level signals through hidden units. All results below use stratified 10-fold cross-validation and report accuracy and macro F1-score.

k-NN Hyperparameter Sweep

Table 1: Digits: k-NN performance for different values of k .

k	Accuracy	Macro F1
1	0.9872	0.9871
3	0.9872	0.9872
5	0.9883	0.9883
7	0.9861	0.9861
11	0.9794	0.9793
15	0.9806	0.9805
21	0.9745	0.9743
31	0.9677	0.9674

Random Forest Hyperparameter Sweep

Table 2: Digits: Random Forest performance for different values of $ntree$.

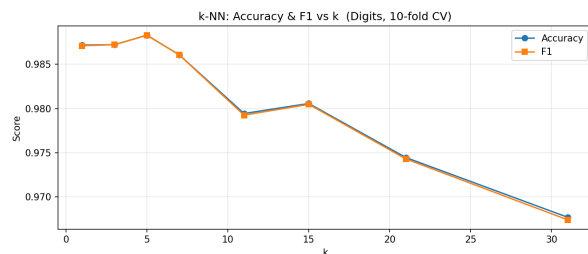
$ntree$	Accuracy	Macro F1
5	0.8965	0.8957
10	0.9377	0.9373
20	0.9605	0.9604
30	0.9621	0.9620
50	0.9694	0.9695
75	0.9716	0.9718
100	0.9660	0.9660
150	0.9749	0.9748

Neural Network Hyperparameter Sweep

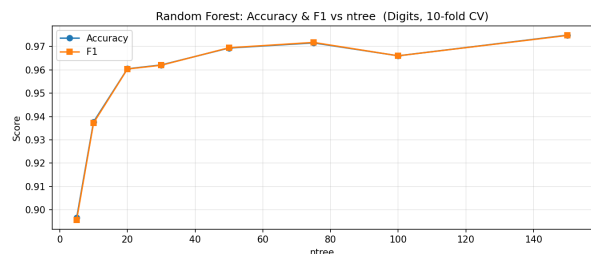
Table 3: Digits: Neural Network performance for selected architectures and regularization values.

Hidden layers	λ	Accuracy	Macro F1
(32,)	0.0	0.9800	0.9798
(64,)	0.0	0.9800	0.9799
(32, 16)	0.0	0.9756	0.9755
(64, 32)	0.0	0.9761	0.9761
(32,)	0.1	0.9544	0.9542
(64, 32)	0.1	0.9538	0.9536

Learning Curves

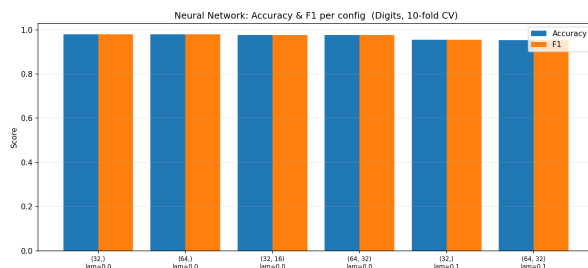


(a) k-NN: Accuracy and F1 vs k

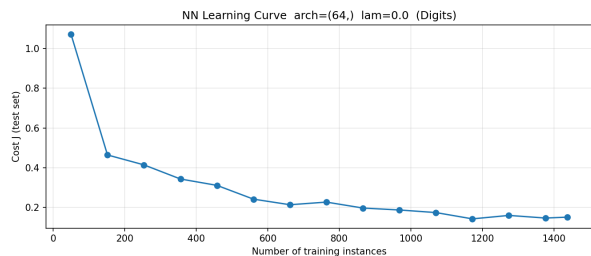


(b) Random Forest: Accuracy and F1 vs $ntree$

Figure 1: Hyperparameter curves for Dataset 1 (Digits).



(a) Neural Network: Accuracy and F1 by configuration



(b) Neural Network: Cost vs training set size

Figure 2: Neural Network graphs for Dataset 1 (Digits).

k-NN achieved the best Digits performance, with $k = 5$ producing 0.9883 accuracy and 0.9883 macro F1. Very small values of k already worked well, which suggests that the normalized pixel space has strong local class structure. Performance declined as k became larger, likely because broad neighborhoods smooth over class boundaries between visually similar digits.

Random Forest performance improved substantially as the number of trees increased, but it remained below k-NN and the Neural Network. This is reasonable for image-like data: an axis-aligned tree split only sees one pixel feature at a time, so the forest needs many trees to approximate

spatial digit patterns. The Neural Network performed strongly, with the best setting at one hidden layer of 64 units and no regularization. Adding regularization with $\lambda = 0.1$ reduced performance, which indicates that these small networks were more likely underfitting than overfitting in this experiment. The learning curve decreased as more training examples were used and then began to flatten, suggesting that the selected network was converging but had less advantage over the simpler k-NN classifier on this dataset.

3.2 Dataset 2 – Parkinson’s Disease

For the Parkinson’s dataset, I evaluated k-NN, Gaussian Naive Bayes, Random Forest, and a Neural Network. The dataset contains 22 numerical voice-measurement features and a binary target variable, **Diagnosis**. k-NN tests whether nearby voice measurements identify similar diagnoses. Naive Bayes provides a simple probabilistic baseline, Random Forests capture nonlinear feature thresholds, and the Neural Network tests whether a learned nonlinear representation improves performance. The F1-score reported here is the F1-score for the positive Parkinson’s class.

k-NN Hyperparameter Sweep

Table 4: Parkinson’s: k-NN performance for different values of k .

k	Accuracy	F1
1	0.9544	0.9676
3	0.9278	0.9530
5	0.9278	0.9528
7	0.9325	0.9565
11	0.8917	0.9319
15	0.8420	0.9034
21	0.8373	0.9028
31	0.8403	0.9047

Gaussian Naive Bayes Hyperparameter Sweep

Table 5: Parkinson’s: Gaussian Naive Bayes performance for different variance floors.

ϵ	Accuracy	F1
1e-9	0.6915	0.7423
1e-7	0.6968	0.7479
1e-5	0.6979	0.7504
0.001	0.6929	0.7497
0.01	0.7129	0.7651
0.1	0.7171	0.7695

Random Forest Hyperparameter Sweep

Table 6: Parkinson’s: Random Forest performance for different values of $ntree$.

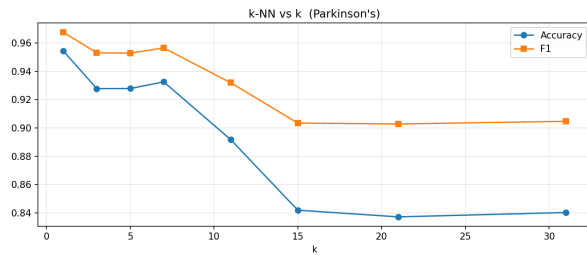
$ntree$	Accuracy	F1
5	0.8554	0.9055
10	0.8817	0.9245
20	0.9014	0.9376
30	0.8878	0.9286
50	0.8920	0.9327
75	0.8823	0.9250
100	0.8928	0.9324
150	0.8911	0.9311

Neural Network Hyperparameter Sweep

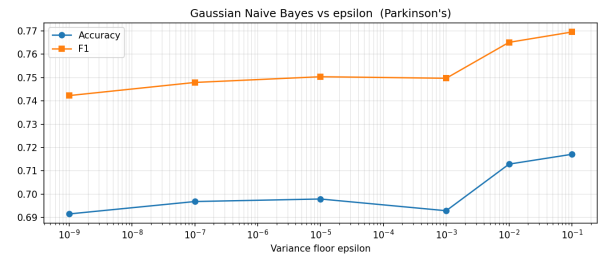
Table 7: Parkinson’s: Neural Network performance for selected architectures and regularization values.

Hidden layers	λ	Accuracy	F1
(8,)	0.0	0.8323	0.8920
(16,)	0.0	0.8378	0.8968
(8, 4)	0.0	0.7648	0.8652
(16, 8)	0.0	0.7659	0.8627
(8,)	0.1	0.8303	0.8966
(16, 8)	0.1	0.7542	0.8598

Learning Curves

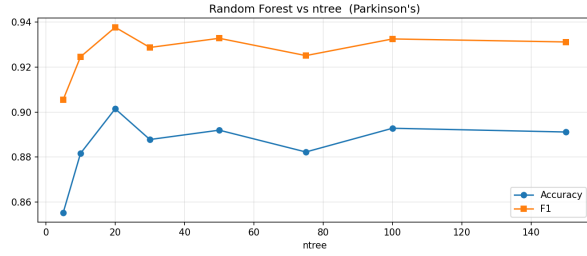


(a) k-NN: Accuracy and F1 vs k

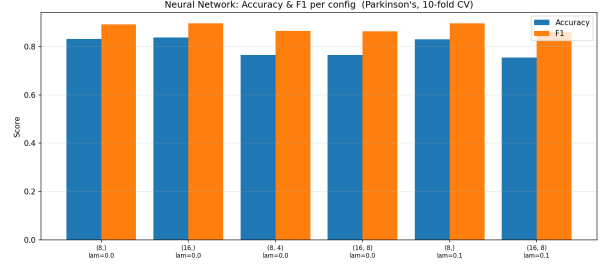


(b) Naive Bayes: Accuracy and F1 vs ϵ

Figure 3: Hyperparameter curves for k-NN and Naive Bayes on Dataset 2.



(a) Random Forest: Accuracy and F1 vs *ntree*



(b) Neural Network: Accuracy and F1 by configuration

Figure 4: Random Forest and Neural Network configuration curves for Dataset 2.

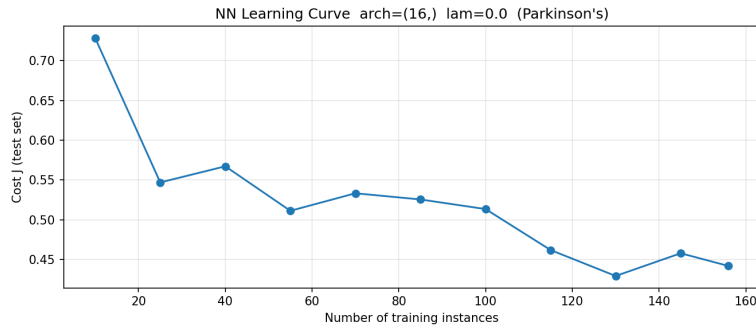


Figure 5: Neural Network learning curve for Dataset 2.

k-NN achieved the best Parkinson's performance, with $k = 1$ producing 0.9544 accuracy and 0.9676 F1. Larger values of k reduced performance, which suggests that the most useful signal is very local in the normalized voice-feature space. Because the dataset is small and imbalanced toward the positive class, larger neighborhoods likely mix less similar patients and smooth away useful diagnostic differences.

Gaussian Naive Bayes was the weakest model. The voice measurements are likely correlated with each other, while Naive Bayes assumes conditional independence given the class. The epsilon sweep shows that increasing the variance floor helped slightly, but it did not close the gap to the other methods. Random Forest was the second-best method: it handled nonlinear feature thresholds better than Naive Bayes, but performance peaked at 20 trees and then varied slightly, which is consistent with the higher variance expected from bootstrap training on only 195 instances. The Neural Network trailed k-NN and Random Forest; the deeper architectures were worse, suggesting that the dataset is too small for the added capacity. Its learning curve generally decreased as more training instances were used, but the curve was not perfectly smooth because the train/test split is small and the training procedure is stochastic.

3.3 Dataset 3 – Rice Grains

To be completed by Akshit.

3.4 Dataset 4 – Credit Approval

To be completed by Akshit.

4 Summary of Results

Table 8: Best performance (Accuracy / F1-Score) per algorithm per dataset. Gray cells indicate the highest value for each metric on each dataset.

Algorithm	Digits		Parkinson's		Rice		Credit	
	Acc	F1	Acc	F1	Acc	F1	Acc	F1
k-NN	0.9883	0.9883	0.9544	0.9676	–	–	–	–
Naive Bayes	–	–	0.7171	0.7695	–	–	–	–
Random Forest	0.9749	0.9748	0.9014	0.9376	–	–	–	–
Neural Network	0.9800	0.9799	0.8378	0.8968	–	–	–	–

5 Discussion

Across the first two datasets, k-NN was the strongest method. This is notable because it is also the simplest model tested. For Digits, the normalized 64-dimensional pixel vectors appear to preserve strong local neighborhoods by class. For Parkinson's, the best value was $k = 1$, which suggests that the strongest signal is concentrated in very near neighbors rather than broad averaged neighborhoods.

Random Forests performed well on both datasets but did not beat k-NN. On Digits, the forest improved as *ntree* increased, but it still had to represent image patterns through axis-aligned splits on individual pixels. On Parkinson's, the forest was useful for nonlinear thresholds but showed some variance across tree counts, likely because the dataset has only 195 instances.

The Neural Network was competitive on Digits but weaker on Parkinson's. The larger Digits dataset gives the network enough examples to learn useful hidden representations. The Parkinson's dataset is much smaller, so deeper networks did not help and likely added unnecessary capacity. Naive Bayes was only evaluated on Parkinson's and performed worst, which is consistent with correlated biomedical voice measurements violating the model's conditional-independence assumption.

6 Extra Credit

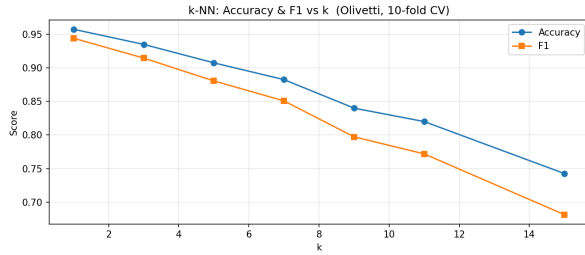
[EC#2] Image Classification on Olivetti Faces

To satisfy the EC#2 criterion of selecting a new *challenging* dataset, I evaluated my k-NN (HW1) and Neural Network (HW4) implementations on the **Olivetti Faces** dataset — 400 grayscale 64x64 images of 40 distinct subjects (10 images per subject), loaded with a `scikit-learn` dataset fetcher. This is an image classification task with 40 classes, far more than any of the four main project datasets. The full notebook is at `notebooks/05_olivetti_ec2.ipynb`.

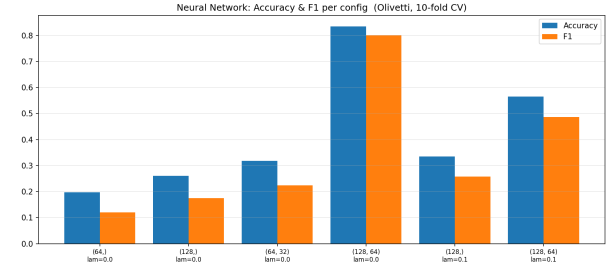
Random Forest was omitted because the from-scratch implementation is too slow with 4096 pixel features. k-NN and Neural Network are also strong face-recognition baselines.

Table 9: Olivetti Faces (EC#2): best result per algorithm, stratified 10-fold CV.

Algorithm	Best Hyperparameter	Accuracy	Macro F1
k-NN	$k = 1$	0.9575	0.9442
Neural Network	(128, 64), $\lambda = 0.0$	0.8350	0.8011



(a) k-NN: Accuracy and F1 vs k



(b) Neural Network: Accuracy and F1 by configuration

Figure 6: Olivetti Faces (EC#2): hyperparameter sweeps.

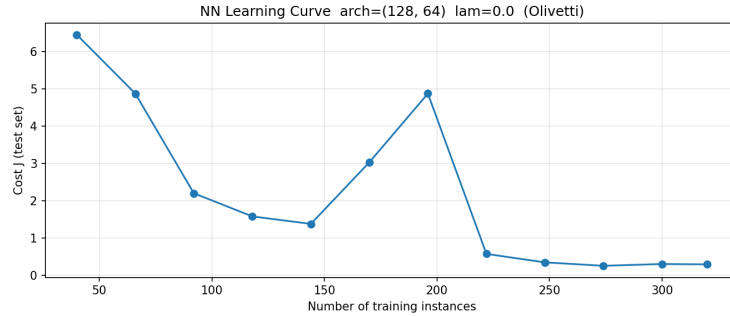


Figure 7: Olivetti Faces (EC#2): NN learning curve.

The Olivetti results support the extra-credit motivation: this is a small but difficult 40-class image-classification task. k-NN achieved the best result, with $k = 1$ reaching 0.9575 accuracy and 0.9442 macro F1. Performance decreased as k increased, which suggests that each subject’s nearest images are highly informative but larger neighborhoods begin mixing other identities. The Neural Network also learned a useful classifier, with its best architecture reaching 0.8350 accuracy and 0.8011 macro F1, but it was weaker than k-NN. This is expected because the dataset has only 400 images and 40 classes, so the network has limited data for learning a stable face representation from 4096 pixel-level inputs.